Adaptive Honeypots for Kubernetes: Automating Calico Network Policies from Attacker Activity

Juhi Sawant *
juhis@vt.edu
Virginia Polytechnic Institute and State University
Blacksburg, Virginia, USA

Clifford Reeve Menezes
reevemenezes@vt.edu
Virginia Polytechnic Institute and State University
Blacksburg, Virginia, USA

Jeewant Choudhry
jeewant5@vt.edu
Virginia Polytechnic Institute and State University
Blacksburg, Virginia, USA

Abstract

Many attacks target cloud applications running on Kubernetes, and static firewall rules often cannot keep up with changing threats. Honeypots are useful here because any traffic they receive is almost certainly malicious and can be used to quickly spot attackers.

This project builds a simple, cloud-ready honeypot system on Kubernetes that can automatically react to such traffic. It runs SSH and HTTP honeypots, watches their logs for malicious IP addresses, and then uses Calico to update cluster-wide network policies so that these IPs are blocked in near real time. The result is a practical way to both observe attacker behavior and automatically cut off repeat attacks with minimal manual effort.

Keywords

Kubernetes, Honeypots, Calico, GlobalNetworkPolicy, Microsegmentation

ACM Reference Format:
Juhi Sawant, Clifford Reeve Menezes, and Jeewant Choudhry. 2026. Adaptive Honeypots for Kubernetes: Automating Calico Network Policies from Attacker Activity. In *Proceedings of Network Security Project (NetSec'25).* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Modern cloud applications increasingly run on Kubernetes and are directly exposed to the Internet, which means a single compromised pod or leaked credential can quickly spread across the cluster if traffic is not tightly controlled [3, 7]. Attackers routinely probe exposed SSH and HTTP services with automated tools [11, 12], and static firewall rules or hand-written policies rarely adapt fast enough to keep up with these attempts [14, 18]. Honeypots help defenders in this setting because they are intentionally vulnerable endpoints with no real users, so any connection attempt is a strong indicator of malicious activity that can be logged and acted on [4, 5, 9].

This project builds a small, practical system that closes the loop between "seeing" and "blocking" such activity in a Kubernetes cluster. It deploys an SSH honeypot based on Cowrie and a simple HTTP honeypot, exposes them using Kubernetes services, and demonstrates interactive attacks via SSH from an external client and from inside the cluster [6, 16]. A Python Policy Enforcer then streams logs from the honeypot pod through the Kubernetes API, extracts the source IPs that appear in failed login attempts, and automatically inserts those IPs into a Calico GlobalNetworkPolicy that denies further traffic from those addresses [10, 13]. In parallel, a basic frontend–backend demo application is deployed in its own namespace, with a Kubernetes NetworkPolicy that only permits traffic from the frontend pod to the backend service, so that normal app traffic stays isolated from honeypot interactions [7, 15]. Together, these components show an end-to-end path where an attacker contacts the honeypot, is detected in the logs, is added to a cluster-wide block list, and then can no longer reach protected services, all without manual intervention.

2 Background

Kubernetes offers both honeypots and fine-grained network policies, but these are usually configured separately and do not react automatically to new attacks [7, 16]. In practice, honeypots generate logs that humans review later [11, 12], while Calico policies stay mostly static [3, 13], leaving a gap between detecting a malicious IP and actually blocking it. This project closes that gap by wiring honeypot logs directly into Calico GlobalNetworkPolicy updates so that attacker IPs are quickly denied across the cluster [14, 15].

2.1 Honeypots in modern networks

Attackers continuously scan the Internet for exposed SSH and web services and then attempt weak passwords or known vulnerabilities with automated tools [11, 12]. In cloud and Kubernetes environments, a single compromised pod can provide a foothold to move laterally and reach other sensitive services if network traffic is not tightly constrained [7, 18].

Honeypots address this by acting as decoy services that should never receive legitimate traffic, so any connection attempt is treated as suspicious and logged for analysis [4, 5, 9]. Cloud honeypot deployments and commercial platforms show that placing such decoys near real workloads helps reveal which ports, protocols, and credentials are actively being targeted in the wild [6, 11].

*All authors contributed equally to this research.

NetSec'25, Blacksburg, VA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

2.2 Microsegmentation in Kubernetes

Kubernetes provides basic support for microsegmentation through NetworkPolicy, which lets operators specify which pods and namespaces are allowed to talk to each other [7]. A default-deny configuration combined with narrow allow rules is widely recommended as a best practice for zero-trust cluster design because it limits how far an attacker can move after an initial compromise [2, 18].

Calico extends this model with its own policy engine and a GlobalNetworkPolicy resource that applies rules across all namespaces and nodes [13]. These cluster-wide policies can be used, for example, to block traffic from known malicious IP ranges at the edge of the cluster, regardless of which workload the attacker is trying to reach [10, 15].

2.3 From passive logs to adaptive policy

Most existing honeypot deployments focus on collecting logs and telemetry that analysts review later [11, 12], while most Kubernetes microsegmentation setups rely on static or slowly changing policy definitions [14, 18]. This separation means there is often a delay between detecting malicious behavior and actually cutting off the attacker at the network level.

The system in this project is designed to close that gap by using honeypot logs as a direct input to Calico's global policies. Honeypot interactions reveal attacker IP addresses, and a controller process converts those observations into immediate updates to a GlobalNetworkPolicy that blocks those IPs cluster-wide, turning passive measurement into active, adaptive defense [10, 13].

3 Methodology

This section describes how the dynamic honeypot system was designed, deployed, and tested on Kubernetes.

```
ubuntu@ip-172-31-28-102:~/dynamic-policy-honeypot-system$ kubectl -n honeypots get svc cowrie-nodeport
NAME              TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
cowrie-nodeport   NodePort   10.43.139.156   <none>        2222:31830/TCP   15m
```

Figure 1: Exposing the Cowrie SSH honeypot via NodePort.

```
(venv) PS C:\Users\reeve\dynamic-policy-honeypot-system> python -m policy_enforcer.main
[*] Loaded local kubeconfig
[*] Existing blocked IPs: set()
[*] Streaming logs from pod: cowrie-honeypot-7b5ddf4447-wld8t
```

Figure 2: Python script to actively monitor logs.

3.1 System architecture

The system consists of three main components deployed on a k3s cluster with Calico networking.

- A honeypot namespace running a Cowrie SSH honeypot and a simple HTTP honeypot service, exposed via a NodePort to receive external attacker traffic [16].
- An application namespace running a toy frontend–backend web application, protected by Kubernetes NetworkPolicy so that only the frontend pod can reach the backend service [7].
- A security namespace hosting a Python-based Policy Enforcer that watches honeypot logs and updates a Calico GlobalNetworkPolicy object that blocks malicious IPs cluster-wide [13].

The creation of the frontend and backend pods, services, and the allow-only-frontend-to-backend NetworkPolicy is shown in the deployment screenshot.

```
PS C:\Users\reeve> ssh test@18.234.50.133 -p 31830
The authenticity of host '[18.234.50.133]:31830 ([18.234.50.133]:31830)' can't be established.
ED25519 key fingerprint is SHA256:0cLPIJ3moxytrf+qQQ5xmKbwfonhocUIg2i/+SI+uZs.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[18.234.50.133]:31830' (ED25519) to the list of known hosts.
test@18.234.50.133's password:
Connection closed by 18.234.50.133 port 31830
```

Figure 3: SSH into Honeypot.

3.2 Honeypot deployment and attack generation

Cowrie was deployed as a containerized SSH honeypot in the honeypot namespace, listening on the standard SSH port inside the cluster and mapped to a high NodePort on the worker node [16]. To simulate an attack, an external client repeatedly attempted to SSH into the honeypot using the exposed node IP and NodePort, generating failed login attempts that Cowrie records in its log files. These interactions are visible in the SSH terminal captures, which show multiple password prompts followed by permission denied responses and, later, the connection being closed once the IP is blocked.

A basic HTTP honeypot (nginx) was also exposed inside the cluster and from other pods to verify that the same pattern — unsolicited requests to the decoy service — would appear in logs and could be treated as malicious.

```
(venv) PS C:\Users\reeve\dynamic-policy-honeypot-system> python -m policy_enforcer.main
[*] Loaded local kubeconfig
[*] Existing blocked IPs: set()
[*] Streaming logs from pod: cowrie-honeypot-7b5ddf4447-wld8t
[+] New attacker IP detected: 10.42.0.1
[+] Added 10.42.0.1 to GlobalNetworkPolicy block-attacker-ips
```

Figure 4: IP logged and blocked in global enforcer

3.3 Policy Enforcer and log processing

The Policy Enforcer is a Python program that uses the Kubernetes API to stream logs from the Cowrie pod in real time [8]. On startup, it loads the current kubeconfig, identifies the honeypot pod, and begins tailing its logs; this behavior is shown in the screenshot where the script reports loading the configuration and starting to stream logs from the Cowrie pod.

Each time the script sees a log line that matches a failed authentication attempt or other attack pattern, it parses out the source IP address. The script maintains an in-memory set of already blocked IPs to avoid duplicate updates, and for each new attacker IP it patches a Calico GlobalNetworkPolicy named, for example, `block-attacker-ips`, adding a new CIDR entry for that IP with an ingress action of deny. The resulting YAML for the GlobalNetworkPolicy confirms that the attacker IP (for example, 10.42.0.1/32) has been added under the `ingress.source.nets` field, with the selector set to `all()` so that the deny rule applies to all pods in all namespaces [10, 13].

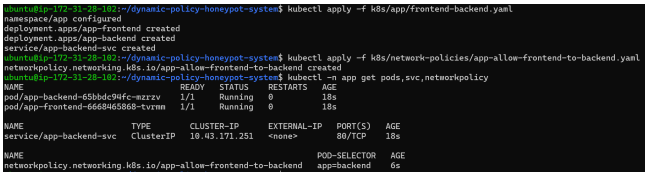

Figure 5: Creating frontend backend pods and doing microsegmentation

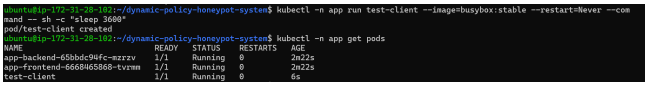

Figure 6: Creating a test client to connect to backend

3.4 Microsegmentation and verification

To demonstrate that normal application traffic remains functional while malicious sources are blocked, a small frontend–backend application was deployed in the app namespace. A NetworkPolicy was applied so that only pods labeled `app=frontend` could initiate connections to the backend service, enforcing basic microsegmentation between application components [7]. A screenshot shows the successful creation of the namespace, deployments, service, and NetworkPolicy, followed by the `kubectl get` output listing the running pods, the backend ClusterIP service, and the policy resource.

From the frontend pod, an HTTP request to the backend service (using `wget` against `app-backend-svc`) confirms that the allowed path works as expected; the response includes the backend's JSON payload and environment information, which appears in the captured terminal output. After the honeypot logs cause the attacker IP to be added to the GlobalNetworkPolicy, a separate test-client pod in the same namespace attempts the same HTTP request to the backend service and receives a "connection refused" error, demonstrating that traffic from the blocked IP range is now denied by Calico even though the backend service itself is still running [13].

Finally, the external SSH client attempts to reconnect to the honeypot after its IP has been added to the block list. The connection is immediately closed after the password prompt, indicating that the Calico policy is preventing further traffic from that source while legitimate cluster-internal paths (such as allowed frontend–to–backend traffic from non-blocked pods) continue to function.

4 Discussion

4.1 Ethical considerations

The system is designed so that all traffic directed at the honeypot is unsolicited and targets a decoy service with no real users or data, which limits privacy and collateral-damage risks. Only technical metadata such as source IP addresses, timestamps, and basic request information is collected from attacker sessions, and this information is used solely to derive blocking rules rather than to profile individuals. The honeypot runs in an isolated namespace with restricted outbound access, reducing the risk that an attacker could compromise it and use it as a pivot to harm third-party systems [4, 9].

4.2 Security benefits and limitations

The main benefit of the approach is that it turns noisy, low-value events such as SSH brute-force attempts into an automatic signal for tightening cluster-wide network policy, without requiring a human to review each log line. This can quickly cut off commodity scanners and repeated probes while keeping legitimate application paths available, as shown by the contrast between successful frontend-to-backend traffic from allowed pods and blocked requests from IPs added to the denylist.

However, the design has limitations: IP-based blocking can be bypassed by attackers who rotate addresses or use large botnets [11], and an unbounded denylist could eventually become hard to manage or impact performance if many thousands of entries are added [18].

4.3 Generality and future extensions

The methodology is not tied to a specific honeypot or CNI implementation; any log-producing honeypot container and any policy engine that exposes an API for updating global rules could adopt a similar "logs-to-policy" controller. In Kubernetes, the same pattern could be extended beyond Cowrie to higher-interaction web or database honeypots, or combined with behavior-based detectors and threat feeds to drive both allow-lists and deny-lists in Calico or other microsegmentation solutions [1]. Future work could also explore adaptive expiration of blocked IPs, richer context (such as attempted commands or URLs) in policy decisions, and integration with zero-trust frameworks so that honeypot-derived signals inform identity- and workload-aware access controls [17].

5 Results and Analysis

5.1 End-to-end blocking workflow

The first set of results verifies that the system correctly detects an attacker, updates Calico policy, and blocks further access. An external client repeatedly attempted SSH logins to the exposed Cowrie service using the node's public IP and NodePort, generating multiple failed password attempts and establishing that the honeypot is reachable from the Internet. While these attempts were occurring, the Policy Enforcer connected to the cluster, loaded the local kubeconfig, identified the Cowrie pod, and began streaming its logs, as shown by the script's startup messages. Shortly after the failed logins, the script reported a "New attacker IP detected" and confirmed that this address had been added to the `block-attacker-ips` GlobalNetworkPolicy, demonstrating successful parsing of logs and dynamic policy update.

Inspection of the Calico policy object shows that the detected attacker IP (for example, `10.42.0.1/32`) appears under the `ingress.source.nets` field, with the selector set to `all()` and the action set to `Deny`, meaning that traffic from that IP is blocked cluster-wide regardless of target namespace or pod [13]. A subsequent SSH attempt from the same external client to the honeypot results in the connection being closed immediately after the password prompt, which confirms that the new rule is being enforced and that repeat SSH attempts from the identified address can no longer reach the service.

5.2 Microsegmentation correctness

The second set of results focuses on verifying that normal application traffic is permitted while unwanted paths are constrained. After deploying the frontend–backend demo application in the `app` namespace, the `kubectl` output shows two running pods, a ClusterIP service for the backend, and a NetworkPolicy allowing only frontend pods to connect to the backend service [7]. From within the frontend pod, an HTTP request issued with `wget` to `app-backend-svc` returns a JSON response with environment and request metadata, demonstrating that the allowed frontend-to-backend path functions correctly under the microsegmentation policy.

To test isolation, a separate test-client pod without the frontend label attempts to reach the same backend service using the same `wget` command. In this case, the request fails with a "connection refused" error even though the backend pod and service are still running, confirming that the NetworkPolicy successfully prevents unauthorized east–west traffic between pods in the same namespace. These results show that the baseline segmentation rules work as intended and that later additions to the GlobalNetworkPolicy do not break legitimate, explicitly allowed application flows.

5.3 Impact of dynamic policy updates

Combining the two behaviors demonstrates the full effect of the dynamic policy system on both external and internal traffic. The honeypot records an SSH attack, the Policy Enforcer adds the attacker IP to the Calico denylist, and the updated GlobalNetworkPolicy immediately begins blocking that source at the cluster edge [13]. At the same time, frontend pods that are not associated with the blocked IP can still communicate with the backend service as permitted by the namespace-scoped NetworkPolicy, while non-frontend pods and any workloads originating from the attacker IP range are denied [7]. Overall, the collected screenshots provide evidence that the system delivers its intended behavior: converting honeypot detections into cluster-wide network enforcement without manual intervention, while preserving correctly configured application traffic.

6 Summary

This project built and evaluated a cloud-native honeypot system that automatically converts attacker activity into Kubernetes network policy updates. A Cowrie-based SSH honeypot and a simple HTTP honeypot were deployed alongside a demo frontend–backend application on a k3s cluster using Calico for networking and microsegmentation. Honeypot logs were continuously streamed by a Python Policy Enforcer, which parsed failed login attempts to identify attacker IP addresses and dynamically inserted them into a Calico GlobalNetworkPolicy denylist affecting all namespaces.

Experimental results showed an end-to-end workflow in which repeated SSH attempts to the honeypot generated log entries, triggered policy updates, and ultimately caused subsequent connections from the same source to be immediately closed, confirming effective cluster-wide blocking. At the same time, Kubernetes NetworkPolicy rules preserved legitimate application paths by allowing only the frontend pod to reach the backend service, while a separate test client and any IPs added to the global denylist were unable to connect. Overall, the system demonstrates a practical way to combine honeypots with Calico's policy engine so that Kubernetes clusters can both observe attacker behavior and react to it automatically, reducing reliance on manual review of logs and static firewall configurations.

7 Related Work

7.1 Honeypots and containerized decoys

Traditional honeypots [5, 9] have long been used to lure attackers away from production systems and to collect detailed traces of malicious behavior for analysis and defense. Recent work has moved this idea into cloud and container environments [6, 12, 16], showing how decoy services can be deployed inside Kubernetes clusters to observe attacks against containerized workloads [11]. Studies of "honeypots in a box" and evaluations of Kubernetes-compatible honeypot platforms such as ContainerSSH highlight that container-based decoys can provide realistic attack surfaces with manageable operational overhead. Compared to these efforts, the system in this project uses relatively simple, low-interaction honeypots (Cowrie and nginx) but focuses on how their logs can directly drive cluster-wide policy changes rather than only serving as a data source for offline analysis.

7.2 Honeypots for proactive defense

Beyond pure monitoring, several works explore using honeypots as part of proactive defense strategies [1, 11], for example to mitigate denial-of-service threats or to generate real-time threat intelligence that feeds blacklists and anomaly detectors. Adaptive honeypot frameworks emphasize [1] dynamic configuration, behavior changes based on attacker actions, and integration with machine learning or policy engines to sustain engagement and enhance detection. Other studies discuss quantifying the effectiveness of dynamic responses once an attack is detected, such as automatically rate-limiting or blocking offending sources [14, 15]. The approach in this project aligns with this proactive line of work but implements a lightweight mechanism tailored to Kubernetes: attacker IPs observed at the honeypot are immediately inserted into a Calico GlobalNetworkPolicy, giving a simple, deterministic response instead of a complex learned policy.

7.3 Microsegmentation and Calico policies

Microsegmentation for Kubernetes has been widely discussed as a core building block for zero-trust security [2, 7], with guidance on using network policies to isolate workloads and restrict east–west traffic between services. Calico extends native Kubernetes NetworkPolicy [3, 13] with richer match conditions, ordered policy tiers, and global policies that apply across all namespaces, and multiple guides describe using these features to implement default-deny baselines and cluster-wide controls [2, 14]. Recent work and vendor material also explore dynamic policy capabilities in Calico, including automatic policy recommendations and fast policy propagation [14, 15] to support real-time network changes. The system presented here builds directly on these ideas but uses a custom controller rather than built-in recommendation engines: instead of analyzing generic flow logs, it specifically consumes honeypot logs

as signals and updates a single GlobalNetworkPolicy object that blocks known attacker IPs cluster-wide.

8 Conclusion

The project demonstrates that integrating honeypots with Kubernetes microsegmentation can provide a simple but effective form of adaptive cluster defense. By deploying Cowrie and HTTP honeypots alongside a segmented frontend–backend application and streaming honeypot logs to a Python Policy Enforcer, the system automatically identifies attacker IPs and inserts them into a Calico GlobalNetworkPolicy denylist. Experiments confirm an end-to-end pipeline: external SSH probes reach the honeypot, are logged, trigger a policy update, and are subsequently blocked, while legitimate frontend-to-backend traffic permitted by namespace-scoped NetworkPolicy remains unaffected.

These results indicate that even modest honeypot deployments can be leveraged to continuously refine cluster-wide network policy, closing the gap between detection and enforcement without complex machine learning or manual rule management. At the same time, the work highlights limitations of IP-based blocking and suggests opportunities for future extensions, such as richer context-aware policies, automated expiration of deny entries, and integration with higher-interaction honeypots and zero-trust identity controls.

9 Future Scope

Future work can extend this prototype along several technical and research directions. First, the current system uses simple IP-based rules; incorporating richer context from honeypot sessions—such as command sequences, URLs, or behavioral fingerprints—could support more precise decisions about when and how to block or throttle traffic, and could feed higher-level threat intelligence systems. Second, the Policy Enforcer could be enhanced with automatic expiry and prioritization of deny entries, preventing unbounded growth of GlobalNetworkPolicy rules and allowing frequently seen or high-risk sources to be treated differently from short-lived scanners.

Third, integrating additional sensors such as Kubernetes audit logs, host-level detection tools, or learning-based classifiers would enable multi-signal adaptive responses, moving beyond pure honeypot triggers to a broader "detect and deceive" pipeline similar to emerging adaptive deception frameworks for Kubernetes. Finally, future work could generalize the architecture to support multiple CNI backends (for example, Cilium as well as Calico), deeper interaction honeypots, and multi-cluster deployments, enabling comparative studies of attacker behavior across different environments and evaluating how automated policy updates affect lateral movement in larger, more realistic systems.

References

[1] A. Alshamrani et al. 2024. A Comprehensive Survey on Cyber Deception Techniques to Improve Cybersecurity. *Computers and Security* (2024).

[2] IBM Corporation. 2025. Controlling Traffic with Network Policies. https://cloud.ibm.com/docs/containers. IBM Cloud Kubernetes Service Documentation, Nov. 2025.

[3] IBM Corporation. 2025. Using Calico Network Policies to Control Traffic on Classic Clusters. https://cloud.ibm.com/docs/containers. IBM Cloud Kubernetes Service Documentation, Oct. 2025.

[4] Inc. CrowdStrike. 2025. What Is a Honeypot in Cybersecurity? https://www.crowdstrike.com/resources/. CrowdStrike Glossary, Aug. 2025.

[5] Inc. Fortinet. 2024. What Is a Honeypot? Meaning, Types, Benefits, and More. https://www.fortinet.com/resources/cyberglossary. Dec. 2024.

[6] N. Krishnan. 2021. Honeypods: Applying a Traditional Blue Team Technique to Kubernetes. https://www.tigera.io/blog/. Tigera Blog, Mar. 2021.

[7] Kubernetes Authors. 2023. Network Policies. https://kubernetes.io/docs/concepts/services-networking/network-policies/. Kubernetes Documentation, Dec. 2023.

[8] Kubernetes Authors. 2023. Policies. https://kubernetes.io/docs/concepts/security/pod-security-standards/. Kubernetes Documentation, Dec. 2023.

[9] Kaspersky Lab. 2020. What Is a Honeypot? How Honeypots Help Security. https://www.kaspersky.com/resource-center. Apr. 2020.

[10] Project Calico. 2022. Use Global Network Policies for Basic Isolation of Future Namespaces. https://github.com/projectcalico/calico/issues/6107. GitHub Issue 6107, May 2022.

[11] Orca Security. 2023. *2023 Honeypotting in the Cloud Report.* Technical Report. Orca Security.

[12] B. Spahn, A. Continella, C. Kruegel, and G. Vigna. 2023. Container Orchestration Honeypot: Observing Attacks in the Wild. In *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID).*

[13] Inc. Tigera. 2021. Calico GlobalNetworkPolicy Resource Reference. https://docs.tigera.io/. Tigera Documentation.

[14] Inc. Tigera. 2025. Enhancing Kubernetes Network Security with Microsegmentation. https://www.tigera.io/blog/. Tigera Blog, Aug. 2025.

[15] Inc. Tigera. 2025. Microsegmentation in 2025: Importance, Types & Best Practices. https://www.tigera.io/resources/. Calico Microsegmentation Guide.

[16] A. Tkachuk. 2025. Creating and Deploying Honeypots in Kubernetes. https://www.apriorit.com/blog/. Apriorit Technical Blog, Oct. 2025.

[17] S. Vashisht and R. Kaur. 2021. Zero Trust Security Model Implementation in Kubernetes-based Cloud Environments. *International Journal of Scientific Research and Applications (IJSRA)* 11, 1 (2021).

[18] S. J. Vaughan-Nichols. 2021. *Micro-Segmentation for Kubernetes Instantiated Ephemeral Workloads.* Whitepaper. SANS Institute.